CampusSwap – Final Reflection Report & Code Contribution

Prepared by Group 1 – SwapSquad: Danial Khan, Payas Hasteer, Suleiman Najim, Keshav Jindal, Badri Widaatalla, Sarah Siddiqui

Date submitted: November 12, 2024

Section 1: Teamwork Reflection

Our team has collaborated effectively to complete each project milestone and executed three effective sprints to ultimately develop the **CampusSwap** web application.

We adopted **Jira** as a project management tool that helped us organize and monitor the progress of tasks. Some issues were encountered in consistently updating Jira and meeting internal deadlines. These issues largely stemmed from a lack of available time and differing schedules; however, tasks were typically completed shortly after deadlines, or compromises were reached. The team's ability to address such concerns ultimately became a strength in our agile process.

The team did not engage in any arguments or major disputes about tasks or project direction. Minor conflicts of interest were resolved with open discussions and fair voting as necessary. This collaborative approach proved to be a valuable team strength.

Overall, our experience highlighted the value of strong communication and accountability in a group setting. Moving forward, we aim to improve our individual accountability to help ensure that each sprint is as effective as possible.

Section 2: Code Contribution

Click on any name to see their assigned PRs: Danial Khan, Payas Hasteer, Suleiman Najim, Keshav Jindal, Badri Widaatalla, Sarah Siddiqui

Note: In Figure 1 below, for Hasteerp, 44,000 lines of code was the first base commit with template code. Additionally, Keshav is not shown since there was a GitHub config issue with his authoring for commits. He has 19 commits in this repository.

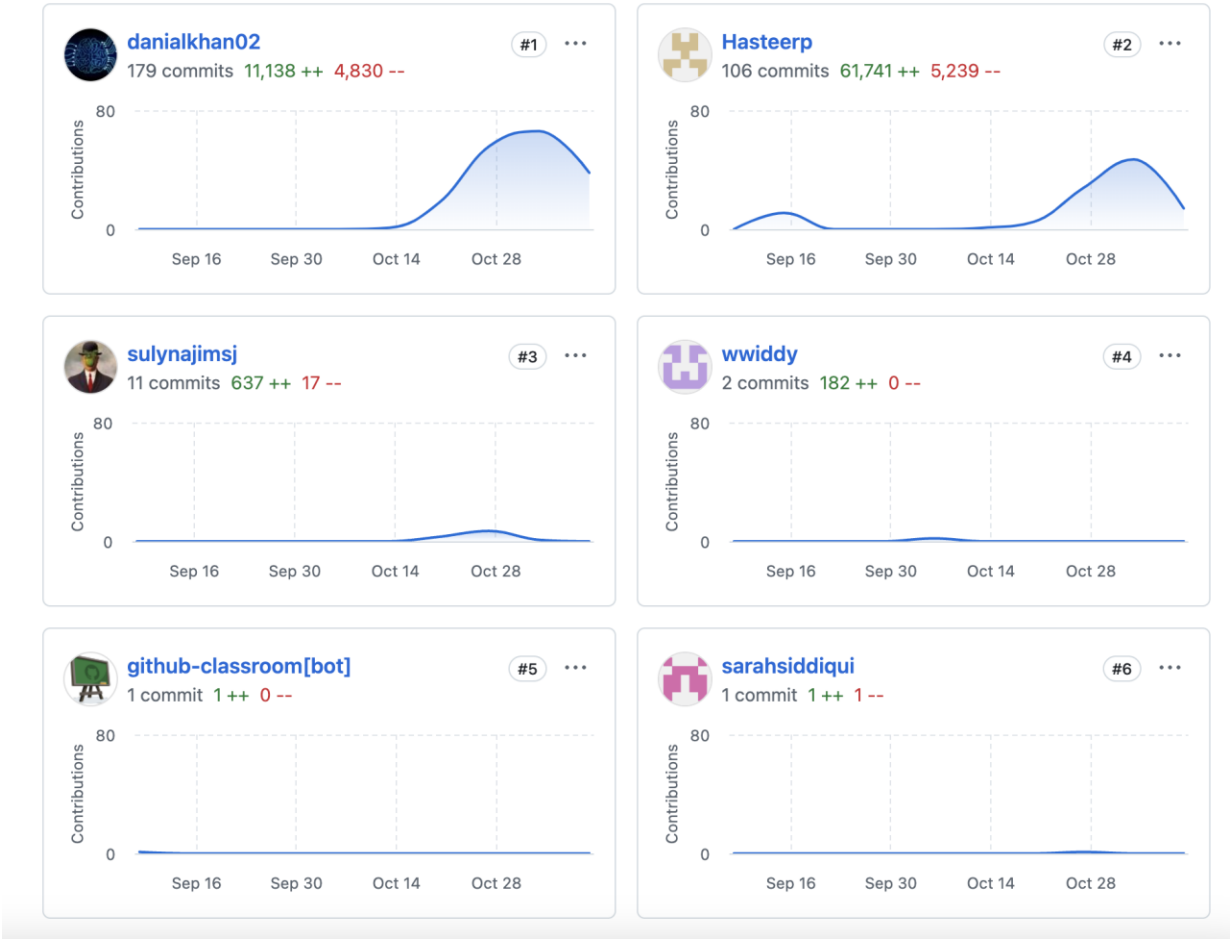

Figure 1: Github Code contribution chart

Section 3: Code Review

We had to adhere to a minimum of 4 components in the checklist below when submitting pull requests. Inserting screen recordings of code/functionality changes along with unit tests was mandatory. To merge our branches into main, each PR had to be approved by another team member– either Daniel or Payas. We incorporated GitHub workflows to run eslint code quality checks, along with running backend and frontend tests in the entire repository each time someone created a pull request. If either eslint or code tests failed, then the branch would not be allowed to merge into main. For front-end code, we employed eslint to ensure consistent coding structure and format across the codebase. We also bifurcated the team into different subgroups based on feature/design implementations. For example, Keshav and Payas collaborated on the login page feature, whilst Badri and Sarah worked on UI/UX design.

- [x] Add a label for this change
- [x] Add a screen recording
- [x] Add unit tests
- [x] Add integration tests
- [x] Update version in Package.json/Pyproject.toml

Figure 2 – PR checklist

We also utilized pair programming, and ensured each team member had their code reviewed during development. We had synchronous and asynchronous code review discussions related to our pull requests. The code review process was compliant with industry standards.

Section 4: Quality Assurance

Unit Testing

For backend unit testing, we decided to use **Pytest**. As a group decision we decided to only test CRUD operations. This is because we use SQLAlchemy Library which automatically converts API models into SQL queries so that testing is not required. The test coverage is available in **Unit Testing Results, Appendix A.**

For front-end unit testing, we decided to use **Jest**. Due to using a existing template from GitHub, we only decided to test the actual visible pages. We decided to test user flow through the page. For example, Homepage would test product listings showing up data on those listings as well as filter, sort, and search button taking inputs. The test coverage is available in **Unit Testing Results, Appendix A.**

Load Testing

After evaluating various tools, we selected **Grafana K6** for load testing, leveraging its capabilities for detailed data visualization across both local and production environments. For initial testing, we focused on the **GetProductList** API, which was anticipated to have the highest data throughput among our CRUD endpoints.

With the initial results **[Grafana Load Testing Results, Appendix A]** parameters, performance fell short of industry standards. Through AI-guided recommendations and debugging, we

implemented Redis caching and pagination, then ran a second test **[Grafana Load Testing Results, Appendix A]**.

Although improvements were noted, high concurrency exposed limitations in the setup, specifically due to the default single-worker configuration in Uvicorn. We reconfigured Uvicorn to run (CPU cores * 2) + 1 workers, allowing parallel request handling **[Grafana Load Testing Results, Appendix A]**.

This configuration met industry standards, reducing response times by **88%**. For production, we deployed AWS t2.xlarge (4 vCPU, 8GB RAM), which supported higher concurrency **[Grafana Load Testing Results, Appendix A]**.

End-to-End Testing

From the very beginning, our team focused on end-to-end testing. This was done by adding screenshots or recordings to each of our PR as it was a part of the PR checklist. This ensured that the team would only push PRs to main once they had tested the flow end-to-end on their local machine. We had a total of 53 PRs with 90%+ PRs with screenshots validating end-to-end testing. This helped to increase the team's confidence in their code merge.

Section 5: Other Contributions

The following are links to UI Design Contributions for applicable members:

Badri Widaatalla Sarah Siddiqui

Appendix A:

Unit Testing Results:

Backend Test Coverage:

Coverage report: 72%

Files | Functions | Classes

coverage.py v7.6.4, created at 2024-11-12 15:21 -0500

File ▲	statements	missing	excluded	coverage
backend/db_interface/chats.py	69	13	0	81%
backend/db_interface/items.py	316	104	0	67%
backend/db_interface/notifications.py	110	40	0	64%
backend/db_interface/seller_feedbacks.py	166	56	0	66%
backend/db_interface/seller_profiles.py	101	31	0	69%
backend/db_interface/users.py	103	44	0	57%
backend/db_models/__init__.py	0	0	0	100%
backend/db_models/base.py	14	1	0	93%
backend/db_models/chat.py	21	0	0	100%
backend/db_models/connection.py	24	6	1	75%
backend/db_models/item_images.py	8	0	0	100%
backend/db_models/items.py	35	0	0	100%
backend/db_models/notifications.py	11	0	0	100%
backend/db_models/seller_feedbacks.py	15	0	0	100%
backend/db_models/seller_profiles.py	13	1	0	92%
backend/db_models/users.py	26	0	0	100%
backend/openai_integration/openai_client.py	45	2	0	96%
Total	**1077**	**298**	**1**	**72%**

coverage.py v7.6.4, created at 2024-11-12 15:21 -0500

Frontend Test Coverage:

File	% Stmts	% Branch	% Funcs	% Lines
All files	56.33	54.63	36.53	57.07
Authentication	0	0	0	0
Login.tsx	0	0	0	0
Logout.tsx	0	0	0	0
Chats	100	50	100	100
ChatPage.tsx	100	50	100	100
HomePage	62.5	51.11	33.33	65.33
HomePage.tsx	62.12	59.09	38.46	65.57
ProductDetails.tsx	64.28	43.47	20	64.28
UserProfile/components	64.64	85.71	45.83	63.91
UserListings.tsx	53.06	71.42	25	52.08
UserWishlist.tsx	76	100	66.66	75.51

Note: we did not test login.tsx and logout.tsx since they're using a 3rd party authentication service, stytch.

Grafana Load Testing Results:

Initial Testing Results:

- **Virtual Users (VUs):** 20
- **Requests:** 764 over 6 mins
- **HTTP Failures:** 0
- **Peak Requests per Second (RPS):** 6.67
- **P95 Response Time:** 7.24s

Second Test Results (Post-Optimization):

- **VUs:** 100
- **Requests:** 979 over 6 mins
- **HTTP Failures:** 0
- **Peak RPS:** 33.33
- **P95 Response Time:** 35.127s

Third Test Results (Increased Worker Count):

- **VUs:** 100
- **Requests:** 7,000 over 6 mins
- **HTTP Failures:** 0
- **Peak RPS:** 32
- **P95 Response Time:** 4.162s

Production Results:

- **Requests:** 8,000 over 6 mins
- **HTTP Failures:** 0
- **Peak RPS:** 36

- **P95 Response Time:** 3.178s